create a partial test double. In testdouble.js, the idea is that it is generally safer and clearer to replace an entire object with a test double rather than have an object exist as part test double, part real object. I can't argue with the theory, though if you're used to RSpec's looser test double style, working with testdouble.js requires some adjustment. As you'll see, the design of the resulting code is to some extent determined by drawing a boundary around logic that is easy to stub together. This is by design; the test is being used to determine the logical structure of the code.

Using testdouble.js happens in three steps: declaring the double, defining its behavior with the td.when method, and optionally verifying its usage after the test has run with the td.verify. Please note that this is not a complete guide to testdouble.js; you're invited to check out the very complete documentation online.[8]

Creating Test Doubles

The testdouble.js library has three methods for creating doubles, which are analogous to RSpec's double method. Which one you use largely depends on whether your JavaScript code is primarily structured around functions, objects, or classes.

If you're trying to replace a bare function, you can use the td.function method. If you call it with no arguments, the result is a bare function suitable for passing as a dependency to code under test.

More frequently, you may want to replace an entire object or class instance. To replace an object, use the td.object function. There are two ways to use td.object. If you don't have a real object but you know the interface of the object you want, you can call td.object with an array of strings, which represent properties of the object that are functions. You then get a test-double object where each of those strings is a test double-fake function, like this:

```
const fake = td.object(["name", "weight"])
fake.name
fake.weight
```

But using td.object with an array of strings has a problem that might be familiar from the discussion of RSpec doubles. If the "real" object's attributes change but the td.object arguments don't, you could have a passing test even though the underlying code is broken. A workaround is to pass td.object an existing object instead:

8. https://github.com/testdouble/testdouble.js